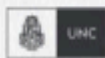


CERTIFICACIÓN UNIVERSITARIA



FCEFYN



Manual para el
alumno

Seguridad Front End



WE WANT
SKILLS!

mundosE

Manual para el alumno: Seguridad Front End.....	2
Objetivo del encuentro:.....	2
Contenido a desarrollar:.....	2
1. XSS, CSRF y clickjacking: qué son y por qué importan.....	3
2. Impacto en aplicaciones modernas y priorización del riesgo.....	4
3. Prevenir XSS en la práctica conceptual: salida contextual, sanitización, CSP y Trusted Types.....	6
4. Prevenir CSRF en la práctica conceptual: tokens, SameSite y Fetch Metadata.....	7
5. Transporte y cabeceras: HTTPS/HSTS, SRI, Referrer-Policy, X-Content-Type-Options, Permissions-Policy, frame-ancestors.....	8
6. Tooling cotidiano en Front End: ESLint (seguridad) y Snyk (SCA/SAST) integrados al flujo.....	10
Bibliografía.....	12

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontrarás un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Seguridad Front End

Objetivo del encuentro:

Implementación teórica y el diseño de buenas prácticas de seguridad en Front End: reconocer y explicar XSS/CSRF/clickjacking y riesgos de cadena de suministro; fundamentar controles del lado del cliente (CSP con nonce + **strict-dynamic**, Trusted Types, tokens anti-CSRF, **SameSite**, Fetch Metadata, **frame-ancestors**), y articularlos con transporte seguro (HTTPS/HSTS), cabeceras, SRI y tooling (ESLint + Snyk) dentro de un SDLC con control en CI/CD.

Contenido a desarrollar:

- ✓ Marco conceptual (cliente, navegador y superficie de ataque).
- ✓ Vulnerabilidades e impacto (XSS, CSRF, clickjacking y supply chain)
- ✓ Prevención de XSS y CSRF (CSP, Trusted Types, sanitización, tokens, SameSite y Fetch Metadata)
- ✓ HTTPS, HSTS, SRI y cabeceras complementarias
- ✓ Tooling de Front End y CI/CD (ESLint + Snyk + políticas de severidad)

1. XSS, CSRF y clickjacking: qué son y por qué importan

La seguridad del Front End se enfrenta a un entorno particular: el **navegador** es un motor de ejecución que no controlamos. En ese contexto, tres familias de problemas aparecen de manera recurrente: **Cross-Site Scripting (XSS)**, **Cross-Site Request Forgery (CSRF)** y **clickjacking**. Comprender su naturaleza teórica permite analizar riesgos y elegir mitigaciones adecuadas sin depender de recetas *ad-hoc*.

XSS engloba fallas que habilitan la **ejecución de JavaScript arbitrario** dentro del contexto de tu aplicación. Las variantes canónicas son: **reflejado**, **almacenado** y **DOM XSS**. Las dos primeras presuponen que un dato controlado por el atacante se “inyecta” en la respuesta del servidor; la tercera se produce completamente en el cliente cuando el código del propio Front End toma datos no confiables (URL, `postMessage`, `localStorage`, respuesta de `fetch`) y los inserta en un **sumidero interpretativo** como `innerHTML`, `insertAdjacentHTML` o un atributo de evento. ¿Por qué esto es grave? Porque el script malicioso heredarán los privilegios de tu aplicación: podrá **leer** contenido sensible en la página, **interceptar** entradas, **llamar** a APIs internas, **modificar** el DOM y **exfiltrar** datos a un servidor bajo control del atacante. Por esa razón, XSS impacta simultáneamente en **confidencialidad**, **integridad** y hasta **disponibilidad** (cuando carga el navegador con tareas no previstas).

CSRF no requiere inyectar código: se apoya en la **autoridad ambiental** que el navegador concede a las **cookies**. Si tu aplicación decide acciones críticas solo porque recibe una cookie válida y **no** verifica la **intención** del usuario ni el **contexto** de la petición, una página maliciosa puede provocar que el navegador emita una solicitud **auténtica** (porque incluye cookies) hacia tu sitio, pero **no intencional**. El resultado típico es la **modificación no autorizada** de recursos (por ejemplo, cambio de email, transferencia de fondos, publicación de contenido), afectando de lleno la **integridad** del sistema. Nótese un punto clave: si una aplicación es vulnerable a **XSS**, un atacante podría simular **CSRF** desde el mismo origen; por eso, el orden conceptual de mitigación sitúa **primero XSS** y **después CSRF**.

Clickjacking (o **UI redressing**) manipula la interfaz en lugar de inyectar código. La página legítima se **embebe** en un `iframe` oculto o superpuesto por la página del atacante; el usuario, sin percibirlo, hace clic donde cree que corresponde, pero su interacción recae en botones o formularios de la aplicación objetivo. El resultado depende de qué acciones puedan dispararse mediante **interacción ciega** y de si existen confirmaciones secundarias. Si pulsar un botón significa “eliminar permanentemente”, el riesgo es evidente. Clickjacking incide en la **integridad** de la interacción humana y puede tener consecuencias indirectas en

confidencialidad o integridad de datos según la función activada.

Estos tres problemas no aparecen en el vacío. En aplicaciones modernas (SPA, microfrontends), su probabilidad de ocurrencia se combina con riesgos de **cadena de suministro** (dependencias npm vulnerables o manipuladas) y con decisiones de arquitectura (uso de `dangerouslySetInnerHTML`, plantillas no escapadas, aceptación de HTML crudo). Por eso, la visión teórica que vas a trabajar es **defensa en profundidad**:

- (1) **prácticas de código** que separen datos de vistas y eviten sumideros peligrosos;
- (2) **políticas del navegador** que restrinjan fuentes y modos de ejecución; y,
- (3) **gobernanza de dependencias** con auditorías continuas.

En síntesis, comprender **qué** es XSS/CSRF/clickjacking, **cómo** operan y **por qué** afectan los objetivos de seguridad (confidencialidad, integridad y disponibilidad) es el paso previo a estudiar mitigaciones como **CSP**, **Trusted Types**, **tokens anti-CSRF**, atributos de **cookies** (**SameSite**, **Secure**, **HttpOnly**), filtros de **Fetch Metadata** y controles como **frame-ancestors**.

Material complementario

- OWASP — XSS: <https://owasp.org/www-community/attacks/xss/>
- MDN — CSRF: <https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/CSRF>
- MDN — Clickjacking: <https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/Clickjacking>

2. Impacto en aplicaciones modernas y priorización del riesgo

Para administrar seguridad en Front End de forma racional, conviene traducir problemas técnicos a **impacto sobre objetivos: confidencialidad, integridad y disponibilidad** (modelo **CIA**). Esta traducción permite priorizar y justificar controles.

Con **XSS**, la **confidencialidad** se ve comprometida cuando un script malicioso extrae contenido de formularios, lee secciones ocultas del DOM o captura tokens no protegidos. La **integridad** sufre cuando ese script modifica la UI, falsifica mensajes, altera el *state* de una SPA o manipula decisiones de negocio (por ejemplo, reemplazar el destinatario de una transferencia en la vista antes de confirmarla). La **disponibilidad** se degrada si el atacante inyecta lazos de trabajo o recursos que ralentizan o bloquean la app. En términos de **exposición**, XSS es ubicuo: cualquier lugar donde se rendericen datos no confiables con un sumidero

interpretativo (por ejemplo, **innerHTML**) constituye una oportunidad de explotación.

En **CSRF**, el vector no es la UI sino el **protocolo**. Si el servidor admite operaciones críticas (no idempotentes) basándose solo en cookies, una página ajena puede dispararlas cuando el usuario visita un dominio malicioso. La **integridad** del sistema queda comprometida porque el acto ejecutado no corresponde a la **intención** del usuario; además, el registro (logs) mostrará una petición genuina, lo que complica la trazabilidad. La severidad se pondera según el alcance de la operación vulnerable (cambiar contraseña vs. modificar preferencias cosméticas).

El **clickjacking** opera sobre la capa de presentación: no ejecuta código propio, pero puede inducir acciones críticas si la aplicación está embebible sin restricciones. El impacto depende de cuán “peligrosos” sean los botones a los que el usuario puede ser engañado para hacer clic y de si existen barreras (confirmaciones secundarias, autenticación adicional).

Para **priorizar** mitigaciones, además del CIA se puede usar un marco como **STRIDE** (suplantación, manipulación, repudio, divulgación, denegación, elevación). XSS se mapea a **manipulación/elevación**; CSRF a **manipulación**; clickjacking a **suplantación** (de la intención del usuario). Este análisis evita sobrerreacciones y ayuda a ordenar las defensas según beneficio/costo.

La arquitectura moderna agrega otra dimensión: **cadena de suministro (supply chain)**. Paquetes comprometidos (por error o malicia) pueden introducir **código en tiempo de build** (scripts de instalación) o **en tiempo de ejecución** (bundle que llega al navegador). Esta vía es silenciosa: un paquete “popular” actualizado automáticamente puede traer comportamientos no intencionales. Por eso, además de prácticas y políticas, es necesario **SCA** (análisis de composición) para cruzar versiones con avisos de seguridad y **SAST** (análisis estático) para detectar flujos peligrosos en el código.

Con este panorama, la **priorización** que vas a usar en el curso es:

- (1) **prevenir XSS** (porque si un atacante ejecuta JS en tu origen, puede “simular” CSRF y muchos otros ataques);
- (2) **reforzar CSRF** (tokens, **SameSite**, validación de contexto con Fetch Metadata);
- (3) **impedir embebidos** no autorizados para evitar clickjacking (**frame-ancestors**); y,
- (4) **asegurar transporte y cabeceras** (HTTPS/HSTS, SRI, **Referrer-Policy**, **X-Content-Type-Options**, **Permissions-Policy**), además de instrumentar **tooling** que evite regresiones (ESLint) y detecte dependencias vulnerables (Snyk u otros).

En síntesis, la evaluación de impacto y la priorización del riesgo te dan un **criterio estable** para decidir: qué arreglar primero, qué controles son obligatorios y qué se monitorea de forma continua en el pipeline.

Material complementario

- OWASP Top Ten: <https://owasp.org/www-project-top-ten/>

3. Prevenir XSS en la práctica conceptual: salida contextual, sanitización, CSP y Trusted Types

La prevención de **XSS** descansa en un principio: **separar datos de código**. En Front End, esto se concreta evitando que datos no confiables alcancen **sumideros interpretativos**. Hay dos herramientas conceptuales: **codificación por contexto** y **sanitización**. La codificación por contexto transforma datos para que sean **literales** en un lugar específico: texto HTML (uso de `textContent` o interpolaciones que escapen), **atributos** (con comillas y escapes adecuados), **URL** (con *percent-encoding* de componentes), **JavaScript embebido** (escapes `\x/\u` cuando sea inevitable) y **CSS**. Cada contexto tiene reglas propias: un string “seguro” en HTML puede no serlo dentro de un atributo o de un `<script>`.

La **sanitización** entra en juego cuando **necesitas interpretar HTML** (por ejemplo, contenido enriquecido enviado por usuarios). Un sanitizador robusto define **listas de elementos/atributos permitidos** y descarta el resto; maneja con cuidado **SVG** y **MathML** (suelen abrir vectores complejos), neutraliza atributos `on*` y restringe **esquemas** peligrosos (`javascript:` o `data:` sin control). La lección teórica es que la sanitización **no** reemplaza la codificación por contexto; son herramientas complementarias.

Sobre esa base, las **políticas del navegador** refuerzan la defensa. **Content Security Policy (CSP)** controla **de dónde** puede cargar y **cómo** puede ejecutar scripts la página. La configuración considerada de “estado del arte” para mitigar XSS es `script-src 'nonce-<aleatorio>' 'strict-dynamic'` sin `unsafe-inline` ni `unsafe-eval`. El **nonce por solicitud** autoriza solo scripts marcados; `strict-dynamic` hereda confianza hacia scripts cargados dinámicamente por uno que ya fue autorizado, eliminando la fragilidad de largas listas de dominios. Además, se recomienda incluir `base-uri 'none'`, `object-src 'none'` y gestionar embebidos con `frame-ancestors`. En la etapa de adopción, **Report-Only** permite observar violaciones sin romper la aplicación y ajustar la política con evidencia.

Trusted Types (TT) aborda **DOM XSS** en su raíz: los sumideros que aceptan cadenas. Al activar `require-trusted-types-for 'script'`, el navegador exige

objetos tipados (por ej., **TrustedHTML**) creados por **fábricas** registradas por la app. De ese modo, una asignación directa a **innerHTML** desde una cadena común **falla** a menos que pase por una *factory* explícita. TT no sustituye CSP: actúan **en capas**. CSP limita fuente y modo de ejecución; TT limita **qué** valores llegan a los sumideros peligrosos.

Otras decisiones complementarias: evitar **eval**, **new Function** y temporizadores con strings (riesgo de **implied eval**); reducir al mínimo **dangerouslySetInnerHTML** en frameworks y rodearlo de garantías; evitar exponer **tokens** en almacenamiento accesible a JS; y preferir **plantillas** o *bindings* que escapen por defecto. En síntesis, la receta conceptual es:

(1) salida por contexto;

(2) sanitización cuando interpretas HTML;

(3) CSP con nonce + **strict-dynamic;**

(4) Trusted Types.

Con esa combinación, la probabilidad de XSS y su impacto se reducen significativamente.

Material complementario

- CSP Cheat Sheet (OWASP): https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html
- CSP Evaluator (Google): <https://csp-evaluator.withgoogle.com/>
- Trusted Types (MDN): <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy/trusted-types>

4. Prevenir CSRF en la práctica conceptual: tokens, **SameSite** y Fetch Metadata

El **CSRF** surge cuando el servidor **confía** en peticiones que traen cookies válidas sin exigir **prueba de intención** ni verificar el **contexto** de la solicitud. La teoría propone tres pilares: **tokens anti-CSRF**, atributos de **cookies** adecuados y validación de **Fetch Metadata**.

Un **token anti-CSRF** es un secreto impredecible asociado a la **sesión/usuario** que el servidor genera y valida. Existen dos patrones: **Synchronizer Token Pattern** (el servidor mantiene el token y lo compara con el recibido) y **double-submit cookie** (el token viaja en cookie y en cuerpo/cabecera y el servidor verifica coincidencia).

Los tokens no deben ser adivinables ni reutilizables fuera de su contexto. Su presencia agrega **intención**: la mera existencia de cookies ya no alcanza para ejecutar acciones sensibles.

Las **cookies** deben llevar **Secure** (solo por HTTPS) y, cuando sea razonable, **HttpOnly** (no accesibles a JavaScript). El atributo **SameSite** modula el envío en contextos entre orígenes: **Lax** bloquea la mayor parte de envíos desde contextos *cross-site* salvo navegaciones de nivel superior de tipo *safe*; **Strict** es aún más restrictivo (puede afectar UX); **None** exige **Secure**. La teoría sugiere comenzar con **Lax** para operaciones normales y endurecer donde aplique.

Fetch Metadata agrega cabeceras como **Sec-Fetch-Site**, **Sec-Fetch-Mode** y **Sec-Fetch-Dest** que describen el **contexto** de la solicitud. El servidor puede rechazar aquellas que **no** provengan del entorno esperado (por ejemplo, bloquear **cross-site** para rutas sensibles). Este filtro, complementario a tokens y **SameSite**, evita que un navegador “bienintencionado” ejecute pedidos que carecen de un contexto válido.

Un refinamiento conceptual consiste en **evitar “simple requests”** para operaciones sensibles: si exigís **Content-Type: application/json** y CORS con **preflight**, navegadores modernos pedirán autorización previa al servidor antes de enviar la solicitud real. Este mecanismo no reemplaza los tokens, pero eleva el costo de explotación.

Hay límites: si existe **XSS** en el mismo origen, el atacante puede forjar token, cabeceras y contexto; por eso se recomienda mitigar **primero XSS**. Asimismo, concentrar credenciales en cookies **HttpOnly** reduce la oportunidad de robo por script. En suma, tu marco mental contra CSRF debe ser: **token como prueba de intención, cookies con SameSite** para contener el envío entre orígenes y **Fetch Metadata** para validar el contexto; en conjunto, disminuyen fuertemente la probabilidad y el impacto.

Material complementario

- CSRF Cheat Sheet (OWASP): https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- Fetch Metadata (web.dev): <https://web.dev/articles/fetch-metadata>

5. Transporte y cabeceras: HTTPS/HSTS, SRI, Referrer-Policy, X-Content-Type-Options, Permissions-Policy, frame-ancestors

La seguridad del Front End no termina en el código. El **transporte** y las **cabeceras** que instruyen al navegador conforman el sustrato donde se apoyan las mitigaciones de XSS/CSRF.

HTTPS proporciona **confidencialidad**, **integridad** y **autenticidad** del canal mediante **TLS**. Su adopción permite marcar cookies como **Secure** y habilita políticas que dependen de **https**. **HSTS** (HTTP Strict Transport Security) agrega una **política persistente** que ordena al navegador usar **solo HTTPS** por un período (**max-age**); con **includeSubDomains**, cubre todo el árbol de nombres; con **preload**, se fuerza HTTPS **desde la primera visita**. El efecto práctico es que el navegador ya no “duda” entre HTTP/HTTPS ni permite al usuario ignorar errores de certificado, reduciendo superficies de ataque por *downgrade*.

SRI (Subresource Integrity) protege **integridad** de recursos externos (por ejemplo, un script en CDN) declarando un **hash** en el atributo **integrity**. Si el archivo no coincide, el navegador lo descarta. SRI traslada parte de la confianza desde el **transporte** a la **inmutabilidad** del artefacto y debe acompañarse de **crossorigin** cuando aplique. En paralelo, **Referrer-Policy** controla cuánta información expone el header **Referer**; una política como **strict-origin-when-cross-origin** reduce la fuga de rutas y parámetros sin romper navegación interna. **X-Content-Type-Options: nosniff** evita que el navegador “adivine” tipos de contenido y ejecute scripts servidos con tipos engañosos. **Permissions-Policy** formaliza el **principio de mínimo privilegio** sobre capacidades (cámara, micrófono, geolocalización) para el documento principal y frames.

Para **clickjacking**, la cabecera actual recomendada es **frame-ancestors** dentro de CSP: define **qué orígenes** pueden embeber el documento. A diferencia de **X-Frame-Options**, es granular y coexistente con otras políticas; la vieja cabecera puede mantenerse como *fallback* para legados. Si tu aplicación necesita **cross-origin isolation** (por ejemplo, para **SharedArrayBuffer** o mitigaciones de **XS-Leaks**), deberás combinar **COOP/COEP** y marcar estáticos con **CORP**.

El **contenido mixto** (documento por HTTPS con subrecursos HTTP) erosiona garantías; navegadores modernos bloquean el **contenido activo** mixto y advierten sobre el **pasivo**. La pauta profesional es **uniformizar**: todo por HTTPS, declarar intenciones con cabeceras, y monitorear violaciones mediante **reportes** (**report-to/report-uri**) para mejorar de manera continua. Esta base no es negociable: si el sustrato de canal/cabeceras es débil, las capas superiores (CSP, TT, tokens) quedan expuestas a by-passes o a una experiencia frágil.

Material complementario

- HSTS (MDN):
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security>

- [SRI \(MDN\):
https://developer.mozilla.org/en-US/docs/Web/Security/Subresource_Integrity](https://developer.mozilla.org/en-US/docs/Web/Security/Subresource_Integrity)
- [Referrer-Policy \(MDN\):
https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy](https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy)

6. Tooling cotidiano en Front End: ESLint (seguridad) y Snyk (SCA/SAST) integrados al flujo

La teoría es efectiva cuando se cristaliza en **herramientas** que evitan regresiones. En un equipo real, necesitas señales automáticas que frenen cambios riesgosos **antes** del deploy.

ESLint es el **linter** dominante para JavaScript/TypeScript. En seguridad, su valor está en bloquear **patrones peligrosos**: `eval`, `new Function`, temporizadores con cadenas (implican ejecución indirecta), y sumideros HTML sin mediación. El plugin **eslint-plugin-no-unsanitized** extiende esta sensibilidad al **DOM XSS**, marcando usos de `innerHTML`/`insertAdjacentHTML` y propiedades similares cuando no hay garantías explícitas. El efecto buscado es que **no** se produzca un *commit* ni un *merge* si persisten violaciones definidas como error. Integrarlo al IDE brinda feedback inmediato; sumarlo a **pre-commit** y **CI** asegura homogeneidad en todo el equipo.

Snyk (u otra solución equivalente) cubre dos espacios: **SCA** (análisis de composición) y **SAST** (análisis estático de seguridad). Con SCA, se comparan tus **dependencias** (incluso transitivas) contra bases de avisos (CVE) y se reportan vulnerabilidades por **severidad**. Con SAST (p. ej., `snyk code test`), se inspeccionan **flujos de datos** en tu código para encontrar fuentes→sumideros peligrosos. En la práctica, estas funciones se integran al pipeline y a los **pull requests**: si hay hallazgos **altos/críticos**, el **status check** falla y la rama no avanza. La gobernanza requiere reglas claras para **excepciones** (quién autoriza, por cuánto, con qué mitigaciones compensatorias) y la generación de **evidencia** (por ejemplo, **SBOM** para saber exactamente con qué se construyó la versión).

Un pipeline saludable muestra **métricas** (tiempo medio de remediación, edad de hallazgos, tendencia por sprint), habilita **comentarios automáticos** en PR con detalle del hallazgo y su localización, y establece **políticas de severidad**: por ejemplo, “no se puede publicar con vulnerabilidades críticas/altas abiertas”. Esta disciplina convierte la seguridad en un **proceso reproducible**, y reduce la dependencia de controles manuales.

En síntesis, tu día a día debe incluir: **(1) ESLint** con reglas de seguridad y el plugin **no-unsanitized** en IDE + CI, **(2) Snyk** (SCA + SAST) en CI con bloqueos por

severidad, **(3) políticas de excepciones** acotadas y **(4) evidencia** (SBOM) para auditoría. Así, la teoría se vuelve práctica sostenible.

Material complementario

- ESLint (docs): <https://eslint.org/docs/latest/>
- eslint-plugin-no-unsanitized (Mozilla):
<https://github.com/mozilla/eslint-plugin-no-unsanitized>
- Snyk CLI: <https://docs.snyk.io/>

Bibliografía

- OWASP Foundation. (2025). *Cheat Sheet Series*. <https://cheatsheetseries.owasp.org/>
- Mozilla. (2025). *MDN Web Docs — HTTP Security*. <https://developer.mozilla.org/>
- Google. (2025). *CSP Evaluator*. <https://csp-evaluator.withgoogle.com/>
- Cure53. (2025). *DOMPurify*. <https://github.com/cure53/DOMPurify>
- Snyk Ltd. (2025). *Documentación Snyk*. <https://docs.snyk.io/>